Laboratório de desenvolvimento de software

Java Swing + Banco de dados

Projeto com banco MySQL

- Primeiramente, precisamos criar uma tabela nova
  - Vamos criar uma tabela veículo que cada pessoa terá um veículo
  - Abra o MySQL Command Line Client

Projeto com banco MySQL

```
mysql> desc pessoa;
+--------+-------------+------+-----+---------+----------------+
| Field  | Type        | Null | Key | Default | Extra          |
+--------+-------------+------+-----+---------+----------------+
| id     | int         | NO   | PRI | NULL    | auto_increment |
| nome   | varchar(50) | NO   |     | NULL    |                |
| sexo   | varchar(1)  | NO   |     | NULL    |                |
| idioma | varchar(1)  | NO   |     | NULL    |                |
+--------+-------------+------+-----+---------+----------------+
4 rows in set (0.00 sec)
```

```
mysql> CREATE TABLE veiculo(
    -> id int AUTO_INCREMENT PRIMARY KEY,
    -> modelo varchar(30),
    -> placa varchar(7),
    -> id_pessoa int NOT NULL,
    -> foreign key(id_pessoa) references pessoa(id)
    -> );
Query OK, 0 rows affected (0.01 sec)
```

Projeto com banco MySQL

- Agora, com a tabela criada, vamos criar uma classe veiculo
  - Onde ela ficará?

Projeto com banco MySQL

- Agora, com a tabela criada, vamos criar uma classe veiculo
  - Onde ela ficará?
    - Qual delas?
  - Precisaremos de uma classe Veiculo dentro do pacote beans que ficarão os atributos (colunas) da tabela
  - E também de um VeiculoDAO dentro do pacote DAO que acessará o banco de dados

Projeto com banco MySQL



Projeto com banco MySQL

```java
public class Veiculo {
    private int id;
    private String modelo;
    private String placa;
    private Pessoa pessoaid;

    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getModelo() {
        return modelo;
    }
    public void setModelo(String modelo) {
        this.modelo = modelo;
    }
    public String getPlaca() {
        return placa;
    }
    public void setPlaca(String placa) {
        this.placa = placa;
    }
    public Pessoa getPessoaid() {
        return pessoaid;
    }
    public void setPessoaid(Pessoa pessoaid) {
        this.pessoaid = pessoaid;
    }
}
```

```java
public class Veiculo {
    private int id;
    private String modelo;
    private String placa;
    private Pessoa pessoaid;

    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getModelo() {
        return modelo;
    }
    public void setModelo(String modelo) {
        this.modelo = modelo;
    }
    public String getPlaca() {
        return placa;
    }
    public void setPlaca(String placa) {
        this.placa = placa;
    }
    public Pessoa getPessoaid() {
        return pessoaid;
    }
    public void setPessoaid(Pessoa pessoaid) {
        this.pessoaid = pessoaid;
    }
}
```

Projeto com banco MySQL

- Agora, vamos criar o VeiculoDAO

```
public class VeiculoDAO {
     private Conexao conexao;
    private Connection conn;

    public VeiculoDAO(){
        this.conexao = new Conexao();
        this.conn = this.conexao.getConexao();
    }
```

Projeto com banco MySQL

```
public void inserir (Veiculo veiculo){
    String sql = "INSERT INTO veiculo (modelo, placa, id_pessoa) VALUES (?,?,?);";

    try {
        PreparedStatement stmt = this.conn.prepareStatement(string:sql);
        stmt.setString(i: 1, string:veiculo.getModelo());
        stmt.setString(i: 2,string:veiculo.getPlaca());
        stmt.setInt(i: 3,il: veiculo.getPessoaid().getId());

        stmt.execute();
    } catch (SQLException ex) {
        System.out.println("Erro ao inserir Veiculo: "+ex.getMessage());
    }
}
```

Projeto com banco MySQL

```
public void editar(Veiculo veiculo){
    try {
        String sql = "UPDATE veiculo set modelo=?, placa=?, id_pessoa=? WHERE id=?";

        PreparedStatement stmt = conn.prepareStatement(string:sql);
        stmt.setString(i: 1, string:veiculo.getModelo());
        stmt.setString(i: 2,string:veiculo.getPlaca());
        stmt.setInt(i: 3,il: veiculo.getPessoaid().getId());
        stmt.setInt(i: 4, il: veiculo.getId());
        stmt.execute();
    } catch (SQLException ex) {
        System.out.println("Erro ao atualizar Veiculo: "+ex.getMessage());
    }
}
```

Projeto com banco MySQL

```
public void excluir(int id){
    try {
        String sql = "delete from veiculo WHERE id=?";

        PreparedStatement stmt = conn.prepareStatement(string:sql);
        stmt.setInt(i: 1, il: id);
        stmt.execute();
    } catch (SQLException ex) {
        System.out.println("Erro ao excluir veiculo: "+ex.getMessage());
    }
}
```

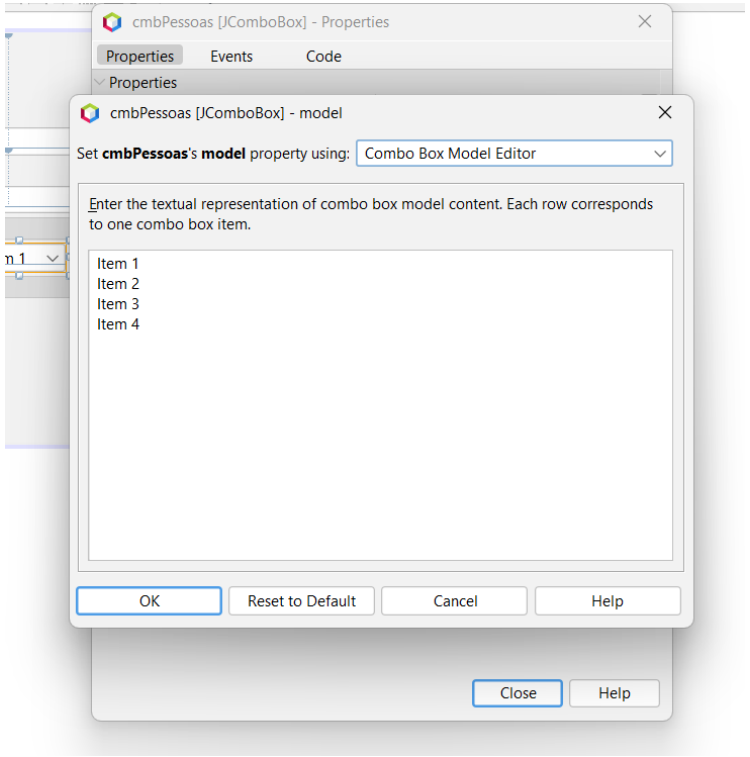
Projeto com banco MySQL

- Agora, vamos criar um formulário para cadastrar o veículo:



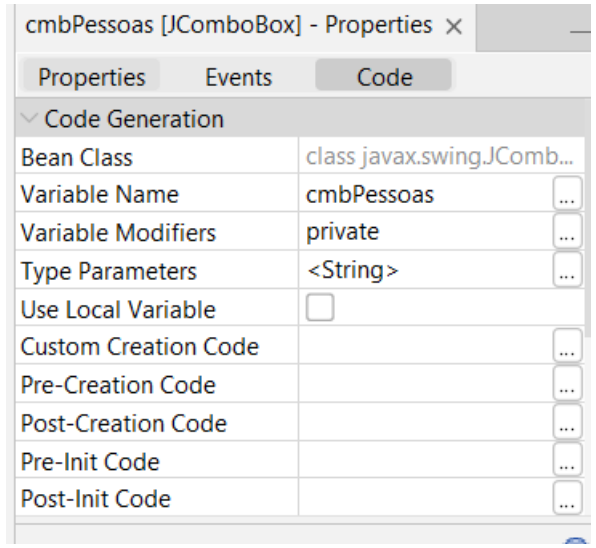
Projeto com banco MySQL

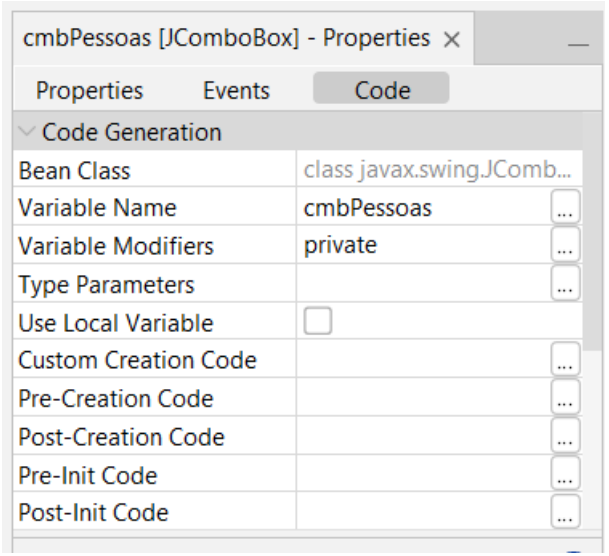
- Limpe todos os dados do combo box pois neste combo serão carregadas as pessoas do banco de dados:



Projeto com banco MySQL

- Precisamos mudar nas propriedades para ele aceitar qualquer objeto dentro do combo box, no caso, um objeto do tipo Pessoa





Projeto com banco MySQL

```
public class FormVeiculo extends javax.swing.JFrame {

    /**
     * Creates new form FormVeiculo
     */
    public FormVeiculo() {
        initComponents();
        preencherComboPessoas();
    }


    public void preencherComboPessoas(){
        PessoaDAO pDAO = new PessoaDAO();
        List<Pessoa> listaP = pDAO.getPessoas();
        for(Pessoa p : listaP){
            cmbPessoas.addItem(item: p);
        }
    }
```

```java
public List<Pessoa> getPessoas(){

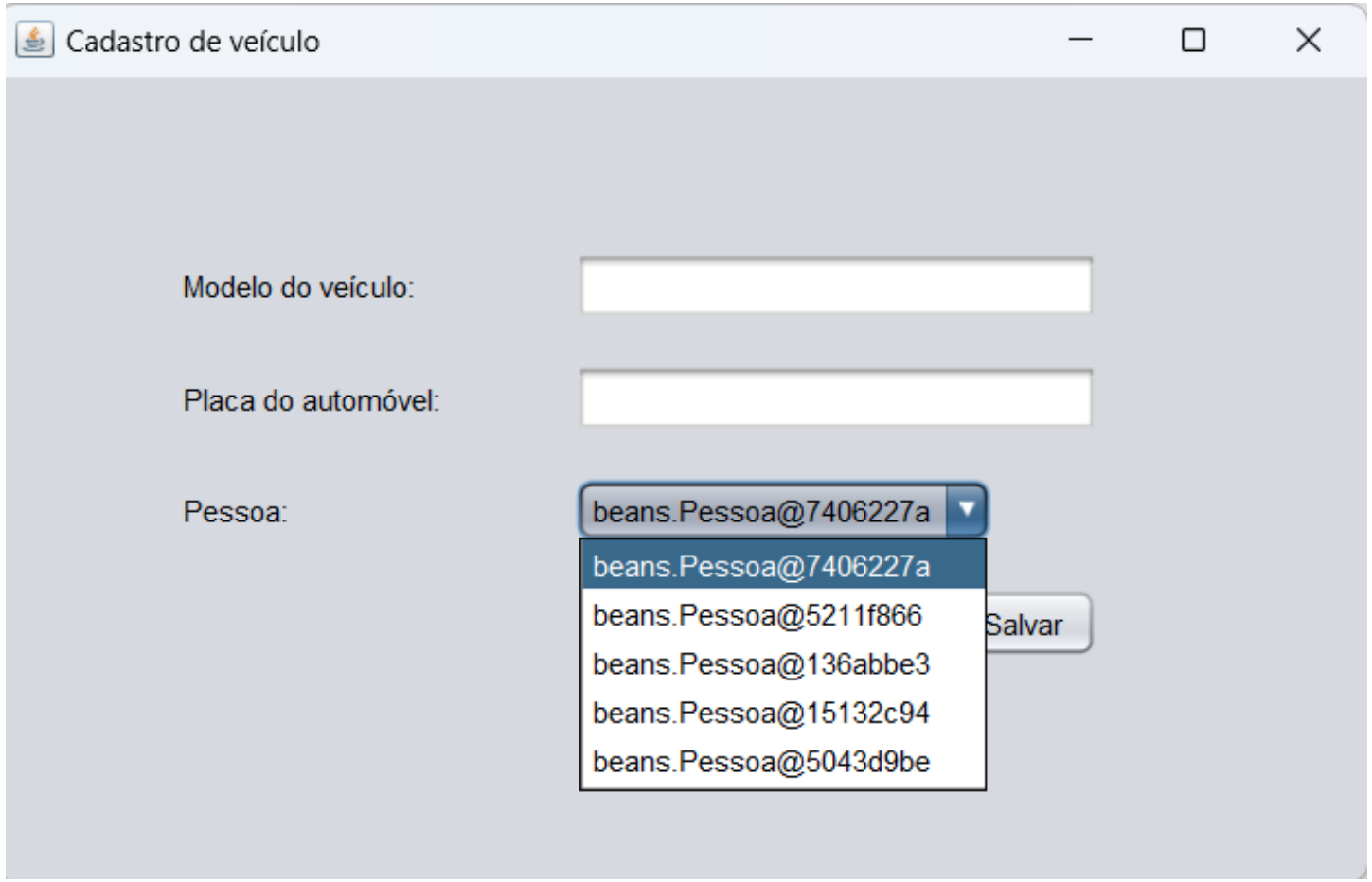
    String sql = "SELECT * FROM pessoa";
    try {

        PreparedStatement stmt = conn.prepareStatement(string:sql,i: ResultSet.TYPE SCROLL INSENSITIVE,
                il: ResultSet.CONCUR_UPDATABLE);

        ResultSet rs = stmt.executeQuery(); //obtenho o retorno da consulta e armazeno no ResultSet
        List<Pessoa> listaPessoas = new ArrayList(); //Preparo uma lista de objetos que vou armazenar a consulta
        //Percorre rs e salvar as informaçoes dentro de um objeto Pessoa e depois adiciona na lista
        while (rs.next()){
            Pessoa p = new Pessoa();
            p.setId(id: rs.getInt(string:"id"));
            p.setNome(nome: rs.getString(string:"nome"));
            p.setSexo(sexo: rs.getString(string:"sexo"));
            p.setIdioma(idioma:rs.getString(string:"idioma"));
            listaPessoas.add(e: p);
        }
        return listaPessoas;
    } catch (SQLException ex) {
        System.out.println("Erro ao consultar todas as pessoas: "+ex.getMessage());
        return null;
    }
}
```

Projeto com banco MySQL

- Funcionou? Ficou estranho né?



Projeto com banco MySQL

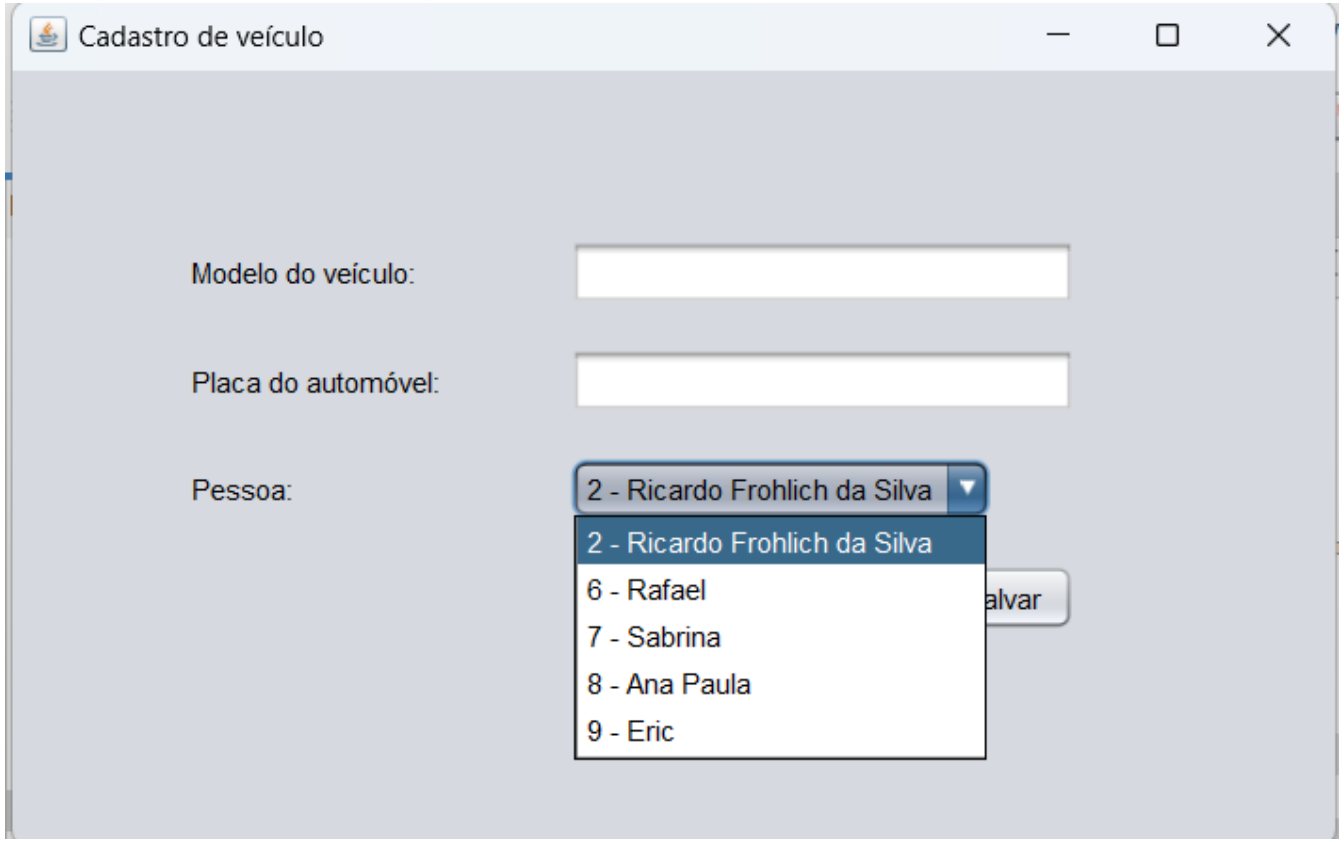
- Vamos melhorar, vá na classe Pessoa e implemente um método toString assim:

```
public String toString(){
    return this.id+" - "+this.nome;
}
```

- E agora?

Projeto com banco MySQL

- E agora?



Projeto com banco MySQL

- Desafio:
  - Tente implementar para adicionar o veículo!

Projeto com banco MySQL

- Dica 1:

```
private void btnSalvarActionPerformed(java.awt.event.ActionEvent evt) {
    System.out.println(x: cmbPessoas.getSelectedItem());
}
```

Projeto com banco MySQL

- Dica 2:

```
private void btnSalvarActionPerformed(java.awt.event.ActionEvent evt) {
    System.out.println(x: cmbPessoas.getSelectedItem());

    Pessoa p = (Pessoa) cmbPessoas.getSelectedItem();

    System.out.println(x: p.getNome());
    System.out.println(x: p.getSexo());
    System.out.println(x: p.getIdioma());
}
```

Projeto com banco MySQL

- Não conseguiram?
  - Está aqui!

```
private void btnSalvarActionPerformed(java.awt.event.ActionEvent evt) {
    Pessoa p = (Pessoa) cmbPessoas.getSelectedItem();
    Veiculo v = new Veiculo();
    v.setModelo(modelo:txtModelo.getText());
    v.setPlaca(placa: txtPlaca.getText());
    v.setPessoaid(pessoaid: p);

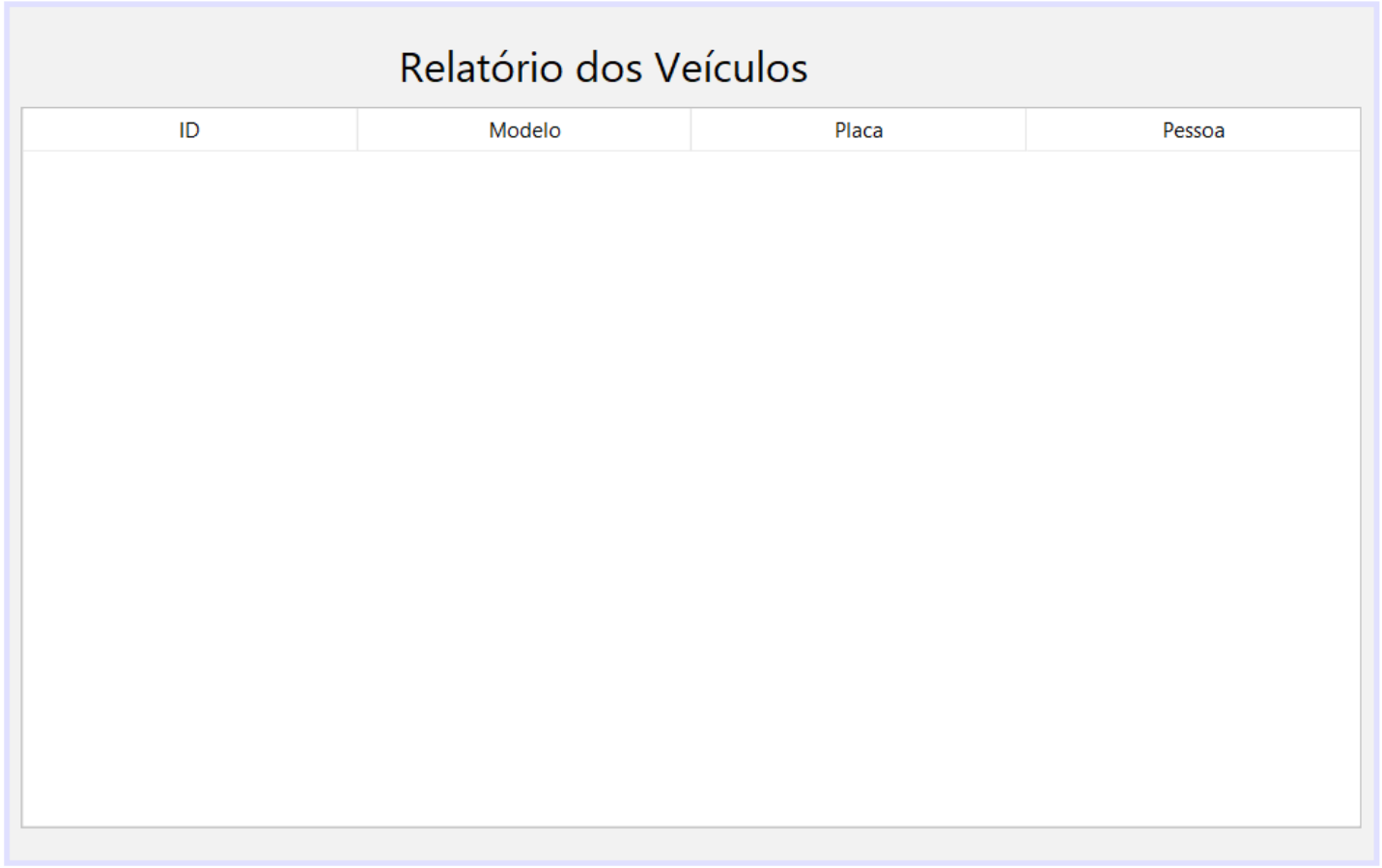
    VeiculoDAO vDAO = new VeiculoDAO();
    vDAO.inserir(veiculo: v);
    limparFormulario();
}
```

Projeto com banco MySQL

```
private void limparFormulario() {
    txtModelo.setText(t: "");
    txtPlaca.setText(t: "");
    cmbPessoas.setSelectedIndex(anIndex: 0);
}
```

Projeto com banco MySQL

- E agora, para apresentar na tela?



Projeto com banco MySQL

```
public List<Veiculo> getVeiculos(){
    String sql = "SELECT * FROM veiculo";
    try {
        PreparedStatement stmt = conn.prepareStatement(string:sql,i: ResultSet.TYPE_SCROLL_INSENSITIVE,
                il: ResultSet.CONCUR_UPDATABLE);

        ResultSet rs = stmt.executeQuery();
        List<Veiculo> listaVeiculos = new ArrayList();

        while (rs.next()){
            Veiculo v = new Veiculo();
            v.setId(id: rs.getInt(string:"id"));
            v.setModelo(modelo:rs.getString(string:"modelo"));
            v.setPlaca(placa: rs.getString(string:"placa"));
            PessoaDAO pDAO = new PessoaDAO();

            Pessoa p = pDAO.getPessoa(id: rs.getInt(string:"id_pessoa"));
            v.setPessoaid(pessoaid: p);
            listaVeiculos.add(e: v);
        }
        return listaVeiculos;
    } catch (SQLException ex) {
        System.out.println("Erro ao consultar todas as pessoas: "+ex.getMessage());
        return null;
    }
}
```

Projeto com banco MySQL

```
public RelatorioVeiculo() {
    initComponents();
    preencheTabela();
}

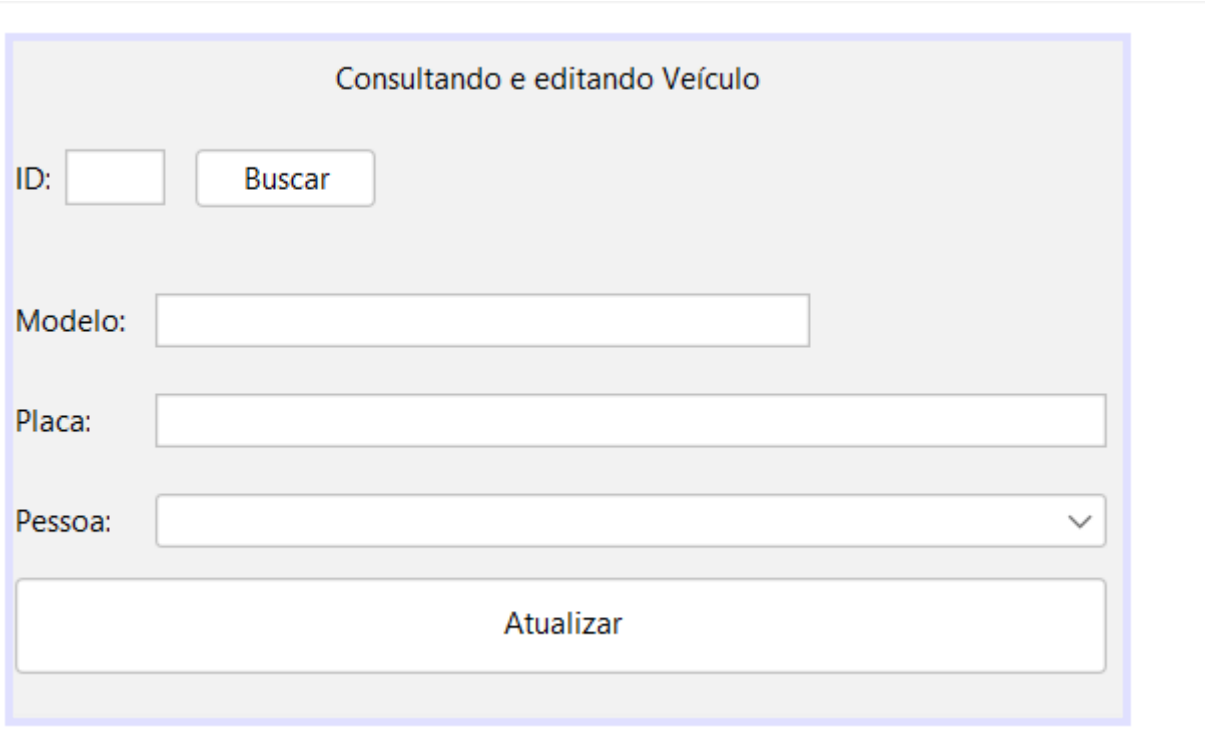
public void preencheTabela(){
    VeiculoDAO vDAO = new VeiculoDAO();
    List<Veiculo> listaVeiculos = vDAO.getVeiculos();
    DefaultTableModel tabelaVeiculos = (DefaultTableModel) tbl_Veiculos.getModel();

    tabelaVeiculos.setNumRows(rowCount: 0);

    for(Veiculo v: listaVeiculos){
        Object[] obj = new Object[]{
            v.getId(),
            v.getModelo(),
            v.getPlaca(),
            v.getPessoaid()
        };
         tabelaVeiculos.addRow(rowData: obj);
    }
}
```

Projeto com banco MySQL

- Criando agora uma tela de consulta e edição de veículo:



Projeto com banco MySQL

```
public class EditarVeiculo extends javax.swing.JFrame {

    public EditarVeiculo() {
        initComponents();
        preencherComboPessoas();
    }
```

```
public void preencherComboPessoas() {
    PessoaDAO pDAO = new PessoaDAO();
    List<Pessoa> listap = pDAO.getPessoas();
    for (Pessoa p : listap) {
        cmbPessoa.addItem(item: p);
    }
}
```

Projeto com banco MySQL

```
private void btnBuscarActionPerformed(java.awt.event.ActionEvent evt) {
    int idVeiculo = Integer.parseInt(s: txtId.getText());
    VeiculoDAO vDAO = new VeiculoDAO();

    Veiculo v = vDAO.getVeiculo(id: idVeiculo);

    if (v == null) {
        limparFormulario();
        JOptionPane.showMessageDialog(parentComponent: this, message: "Veiculo não encontrado");
    }
    else {
        txtId.setText(idVeiculo + "");
        txtModelo.setText(t: v.getModelo());
        txtPlaca.setText(t: v.getPlaca());

        Pessoa pessoaSelecionada = (Pessoa) v.getPessoaid();
        cmbPessoa.setSelectedItem(anObject: pessoaSelecionada);
    }
}
```

Projeto com banco MySQL

```
public Veiculo getVeiculo(int id){
    String sql = "SELECT * FROM veiculo WHERE id = ?";
    try {
        PreparedStatement stmt = conn.prepareStatement(sql,resultSetType: ResultSet.TYPE_SCROLL_INSENSITIVE,
                resultSetConcurrency: ResultSet.CONCUR_UPDATABLE);

        stmt.setInt(parameterIndex: 1, x: id);
        ResultSet rs = stmt.executeQuery();
        Veiculo v = new Veiculo();

        rs.first();

        v.setId(rs.getInt(columnLabel: "id"));
        v.setModelo(rs.getString(columnLabel: "modelo"));
        v.setPlaca(rs.getString(columnLabel: "placa"));
        PessoaDAO pDAO = new PessoaDAO();

        Pessoa p = pDAO.getPessoa(rs.getInt(columnLabel: "id_pessoa"));
        v.setPessoaid(p);
        return v;

    } catch (SQLException ex) {
        System.out.println("Erro ao consultar pessoa: "+ex.getMessage());
        return null;
    }
}
```

Projeto com banco MySQL

```
private void btnAtualizarActionPerformed(java.awt.event.ActionEvent evt) {
    Veiculo v = new Veiculo();
    v.setId(id: Integer.parseInt(s: txtId.getText()));
    v.setModelo(modelo:txtModelo.getText());
    v.setPlaca(placa: txtPlaca.getText());
    v.setPessoaid((Pessoa) cmbPessoa.getSelectedItem());

    VeiculoDAO vDAO = new VeiculoDAO();
    vDAO.editar(veiculo: v);
    limparFormulario();
}
```

Projeto com banco MySQL

- Funcionou?
  - Provavelmente não, pois para isso, precisamos reimplementar o método equals
  - O método equals() em Java é usado para comparar se dois objetos são considerados "iguais".
  - Por padrão, o método equals() da classe Object (que todas as classes herdam) verifica se as referências dos objetos são as mesmas, ou seja, se eles estão apontando para o mesmo objeto na memória.

Projeto com banco MySQL

```java
public class Pessoa {

    private int id;
    private String nome;
    private String sexo;
    private String idioma;

   @Override
    public boolean equals(Object obj) {
        if (this == obj) {
            return true; // Se for o mesmo objeto, retorna true
        }
        if (obj == null || getClass() != obj.getClass()) {
            return false; // Se o objeto for nulo ou de classes diferentes, retorna false
        }
        Pessoa pessoa = (Pessoa) obj;
        return id == pessoa.id; // Compara os ids
    }
```

Projeto com banco MySQL

- E agora?

Atividade

- Agora, no sistema que vocês estão trabalhando com alunos e professores, faça:
  - Uma nova tabela chamada Disciplina e nesta tabela deverá ter os atributos nome e carga horária e uma chave estrangeira que referencie a professores
  - Crie um cadastro de Disciplinas que busque os professores e salve no banco de dados
  - Faça também tela de edição para Disciplina que possa alterar o professor cadastrado.

Atividade 2

- Já fez aquele?
  - Ou se preferir, tenho outro, no próximo slide:

Atividade 2

- Crie uma tabela chamada Categorias com os seguintes atributos:
  - id: chave primária, auto-incrementável.
  - nome
- Crie uma tabela chamada Produtos com os seguintes atributos:
  - id: chave primária, auto-incrementável.
  - Nome
  - Preco
  - Quantidade
  - categoria_id: chave estrangeira

Atividade 2

- Implemente um formulário que permita o cadastro de novos produtos.
- O formulário deve incluir a possibilidade de selecionar a categoria do produto, listando as categorias cadastradas.
- As operações de CRUD (Criar, Ler, Atualizar, Deletar) devem ser implementadas para o cadastro de produtos.
- O sistema deve permitir que a categoria de um produto seja alterada durante a edição.
- Desenvolva uma tela que exiba todos os produtos cadastrados, mostrando o nome do produto, o preço, a quantidade em estoque e a categoria associada.
- Implemente uma tela separada para cadastrar, editar e listar as categorias.
- Adicione uma funcionalidade de busca que permita procurar produtos por nome ou categoria.